



Los Sin Chamba

Integrantes: Lautaro Martinez, Gabriel Maculus, Leandro Orozco, Kevin Castilla, Mariano Rasgido, Ezequiel Diaz, Samira Baz, Jose Rodriguez.

Introducción al análisis de datos

Actividad n° 1 : (en grupo)

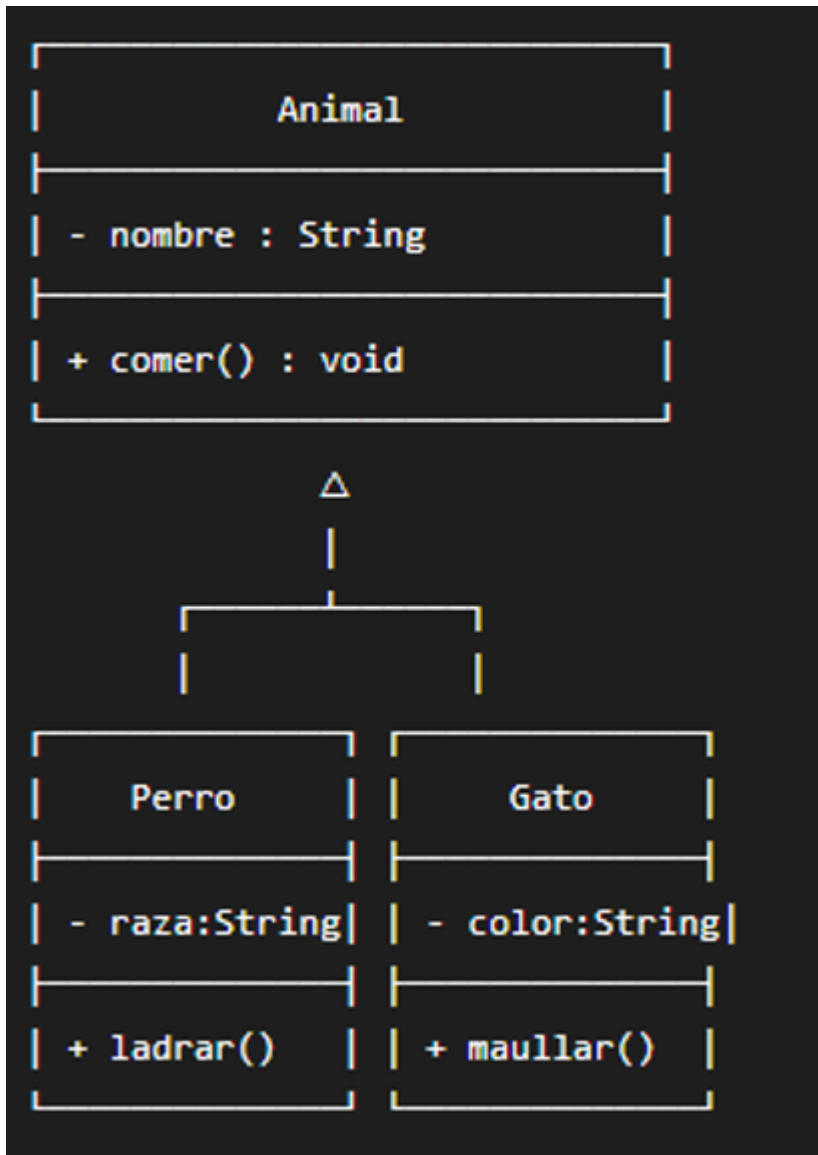
·Imaginemos un programa que gestiona diferentes tipos de animales utilizando polimorfismo para representar comportamientos comunes como comer y sonido.

Actividad n°2: (en grupo)

Actividad: Los siguientes diagramas UML representan diferentes situaciones de Poo. Cada diagrama contiene un error. Analizá cada caso, identifica el error y realiza las modificaciones necesarias para obtener un diagrama UML correcto. Luego explica qué pilar de POO se aplica en cada caso.

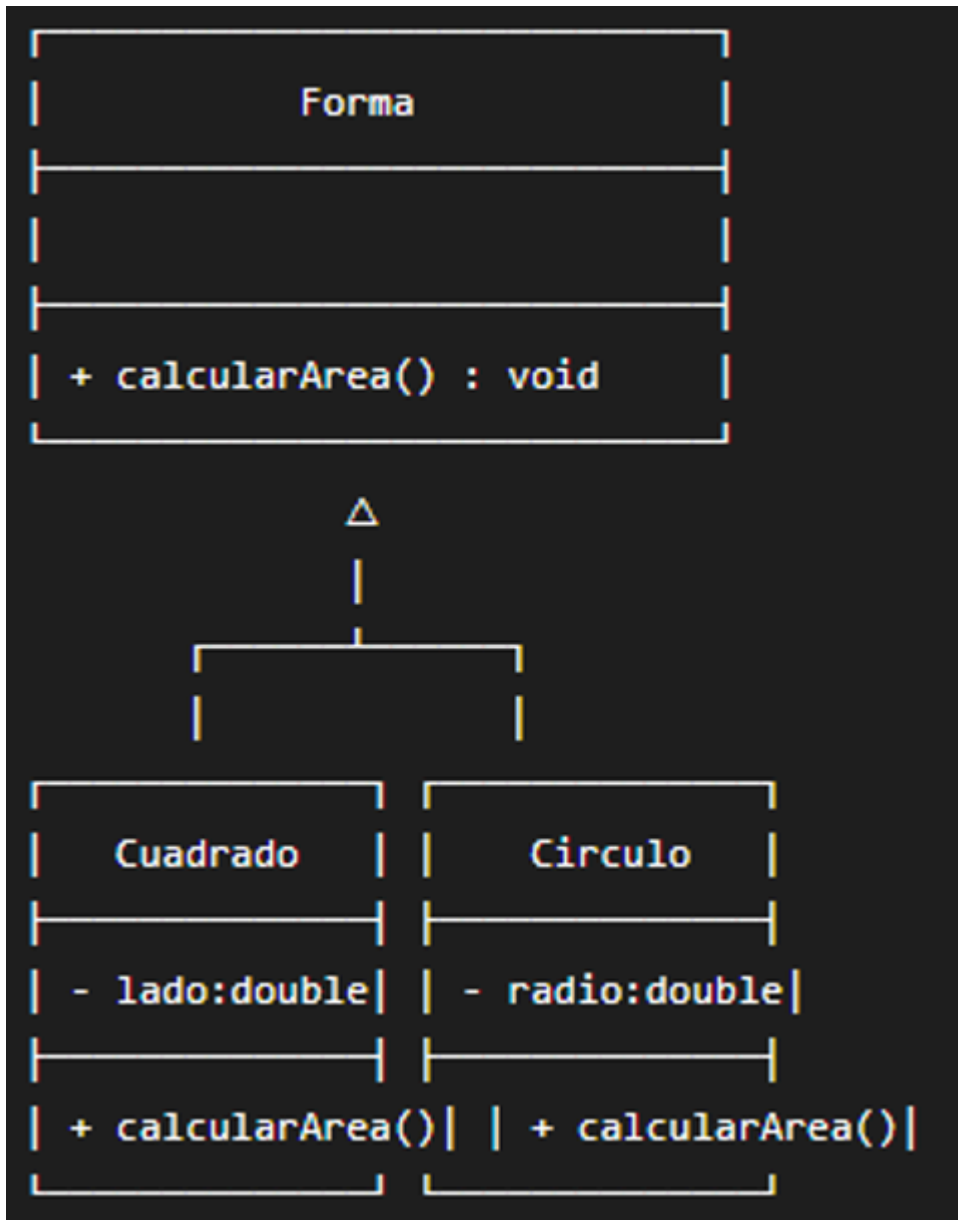
Caso 1: Animales

1)Revisar si las relaciones de generalización están correctamente representadas y completar/corregir lo que corresponda.



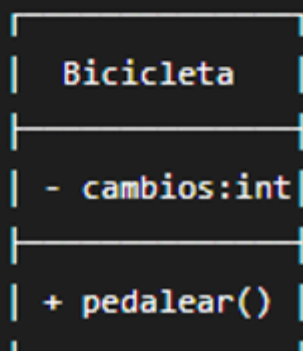
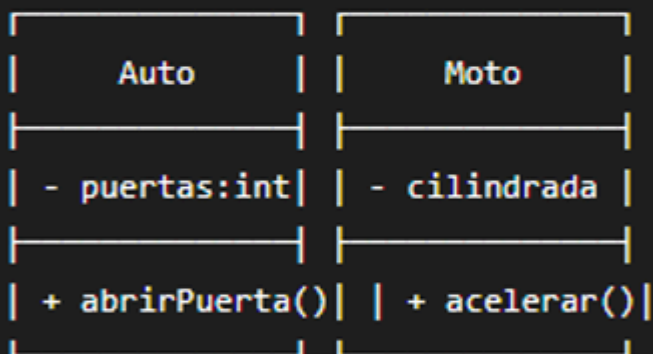
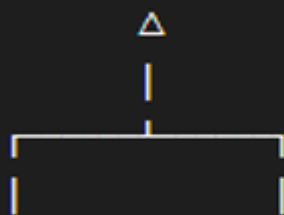
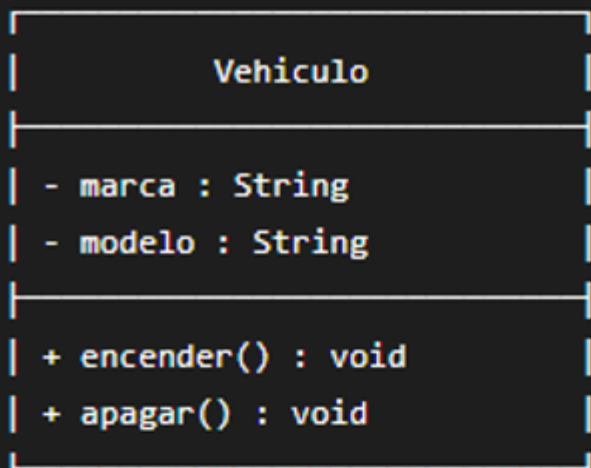
•Caso 2 : Formas

·1) Determinar cómo debe declararse `calcularArea()` en Forma para que las clases hijas puedan implementar su propio comportamiento y exista polimorfismo.



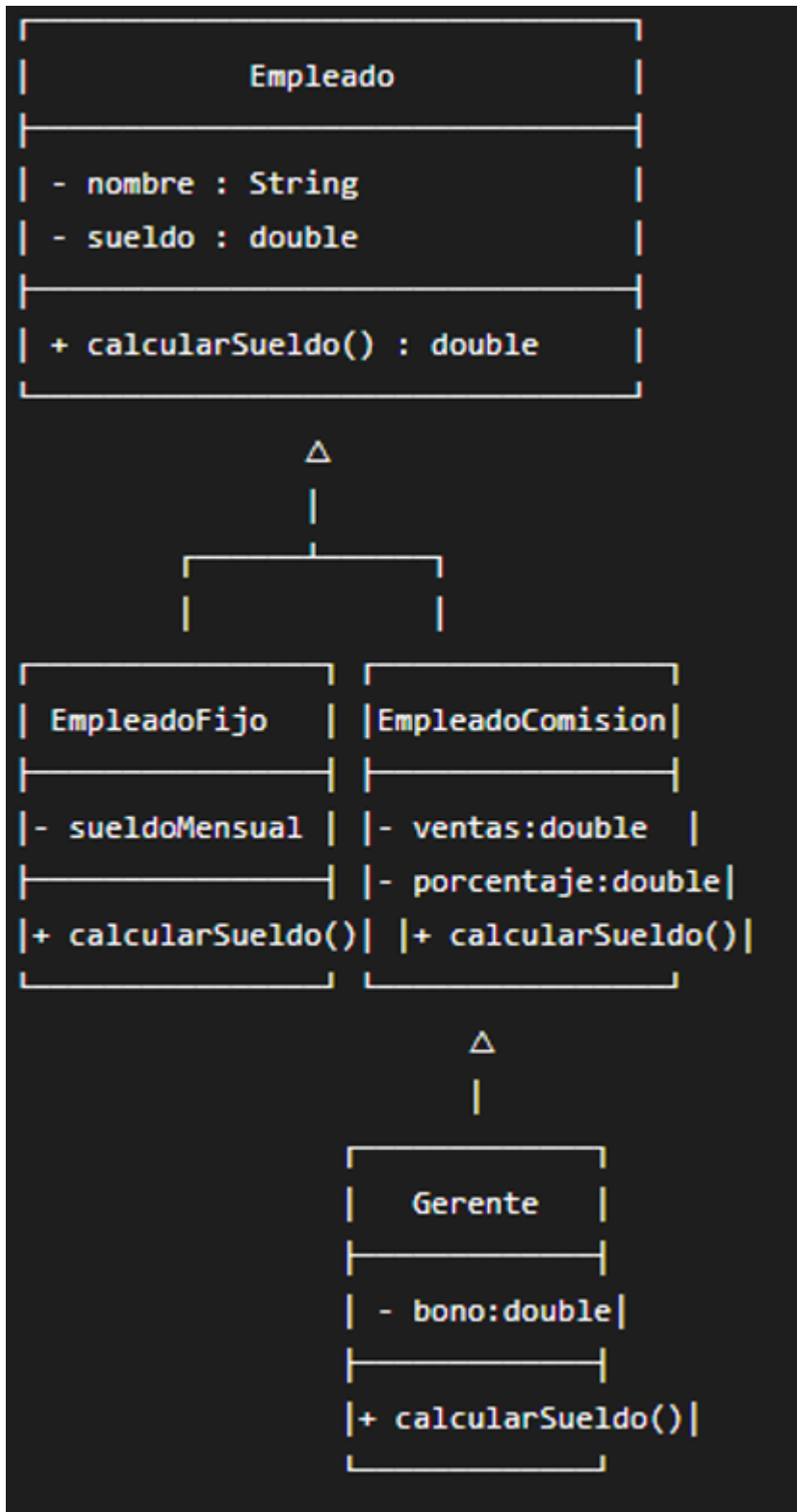
·Caso 3: Vehículo

- 1) Revisar donde hay que corregir la relación de herencia



·Caso 4 : Empleado

·1) Revisar dónde está el error



Resolución de la Actividad N° 2

Caso 1: Animales

- **Error:** En el diagrama, la relación de herencia está representada al revés. En UML, la flecha de generalización (el triángulo vacío) debe apuntar desde las clases hijas hacia la clase padre, no al revés.

- **Modificación:** Invertir la dirección de las flechas. Deben salir de **Perro** y **Gato** y apuntar hacia la clase **Animal**.
- **Pilar de POO: Herencia.**

Caso 2: Formas

- **Error:** El método **calcularArea()** en la clase **Forma** está declarado para retornar **void**. Un cálculo de área necesita devolver un valor (por ejemplo, un **double**). Además, para que exista polimorfismo genuino, este método debe ser abstracto en la clase base para obligar a las clases hijas a implementarlo.
- **Modificación:** Cambiar el tipo de retorno a **double** (quedaría **+ calcularArea(): double**). En UML, para indicar que es un método abstracto, su nombre debe escribirse en *cursiva*.
- **Pilar de POO: Abstracción y Polimorfismo.**

Caso 3: Vehículo

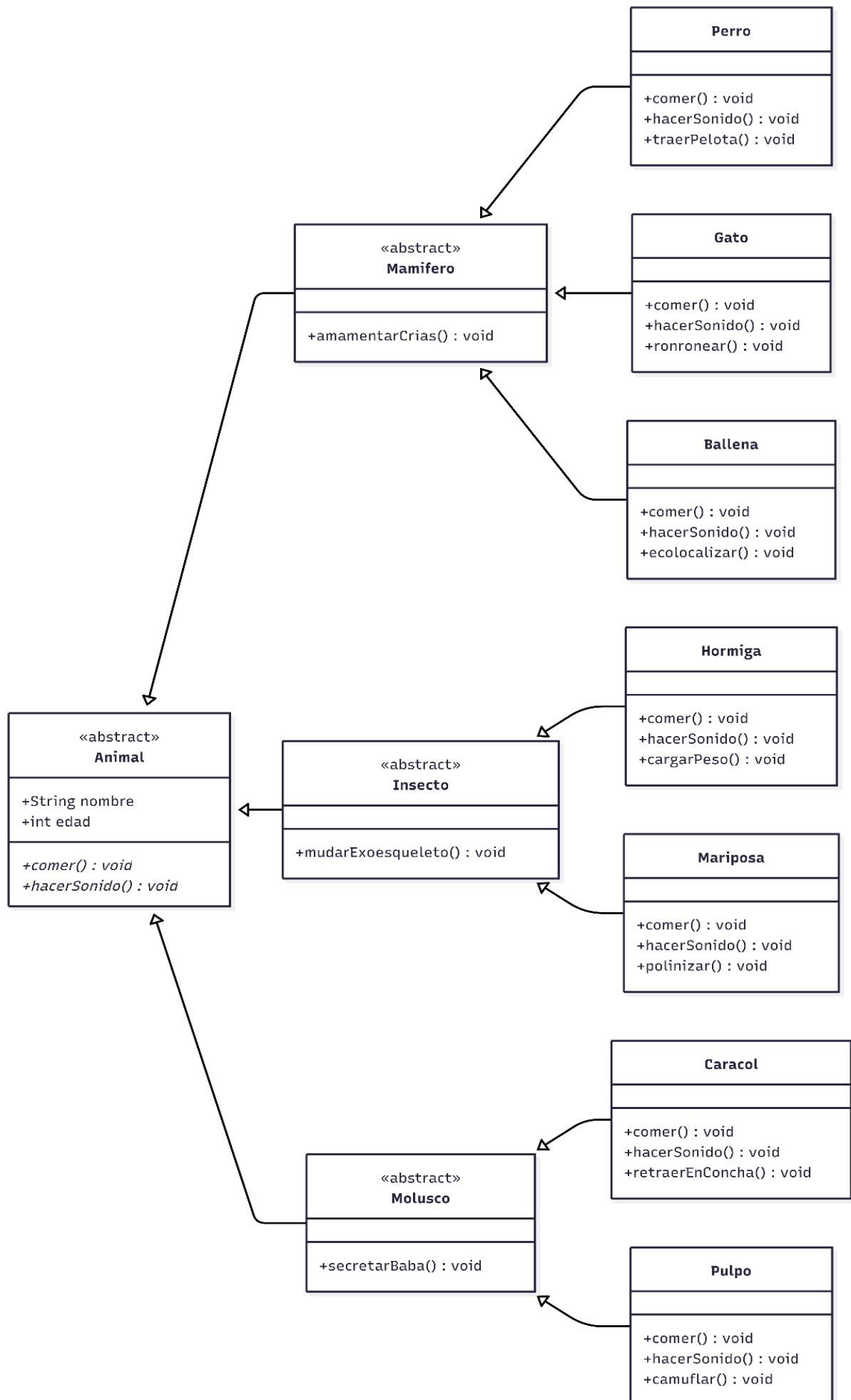
- **Error:** Hay un error conceptual en la jerarquía: la clase **Bicicleta** está heredando de **Moto**. Una bicicleta no es un tipo de motocicleta.
- **Modificación:** Eliminar la flecha de herencia entre **Bicicleta** y **Moto**. La clase **Bicicleta** debe heredar directamente de la clase principal **Vehiculo**.
- **Pilar de POO: Herencia.**

Caso 4: Empleado

- **Error:** La clase **Gerente** hereda de **EmpleadoComision**. Estructuralmente esto es incorrecto, ya que un gerente no es una derivación o un subtipo lógico de un empleado que cobra por comisión.
- **Modificación:** La clase **Gerente** debería heredar directamente de la clase principal **Empleado**.
- **Pilar de POO: Herencia** (por la corrección de la jerarquía) y **Polimorfismo** (ya que vemos que **Gerente** implementa su propia versión del método **+ calcularSueldo()**, sobrescribiendo el comportamiento de la clase padre).

Actividad nº 1 : (en grupo)

•Imaginemos un programa que gestiona diferentes tipos de animales utilizando polimorfismo para representar comportamientos comunes como comer y sonido.



1. El Diseño en MongoDB (Patrón Polimórfico)

En lugar de crear una tabla o colección separada para cada animal, agrupamos todos los documentos en una sola colección llamada `animales`.

Para lograr el **polimorfismo**, todos los documentos compartirán campos base (como el `nombre`), pero tendrán un campo clave llamado **discriminador** (por ejemplo, `tipo`), y campos específicos que representan cómo cada uno cambia su comportamiento al comer o el sonido que hace.

2. Insertando los documentos (Los Objetos)

Podemos insertar los diferentes animales en la colección usando `insertMany`. Nota cómo cada documento adapta las acciones a su especie:

JavaScript

```
db.animales.insertMany([
  {
    "nombre": "Olivia Luppe",
    "tipo": "Perro",
    "comportamiento": {
      "sonido": "¡Guau! ¡Guau!",
      "comer": "Alimento balanceado"
    }
  },
  {
    "nombre": "Salem",
    "tipo": "Gato",
    "comportamiento": {
      "sonido": "¡Miau!",
      "comer": "Pescado"
    }
  },
  {
    "nombre": "Lola",
    "tipo": "Vaca",
    "comportamiento": {
      "sonido": "¡Muuu!",
      "comer": "Pasto del campo"
    }
  }
])
```

3. El Polimorfismo en Acción (La Consulta)

Cuando tu aplicación necesite que "todos los animales coman o hagan su sonido", no le importa qué tipo de animal es (abstracción). Simplemente hace una consulta general a la colección apuntando a los campos comunes.

Por ejemplo, si hacemos un `find()` para listar cómo se comunica cada uno:

JavaScript

```
db.animales.find(
  {}, // Traemos todos los animales
  {
    "_id": 0,
    "nombre": 1,
    "tipo": 1,
    "comportamiento.sonido": 1
  }
)
```

Resultado de la consulta:

JSON

```
[
  { "nombre": "Olivia Luppe", "tipo": "Perro", "comportamiento": { "sonido": "¡Guau! ¡Guau!" } },
  { "nombre": "Salem", "tipo": "Gato", "comportamiento": { "sonido": "¡Miau!" } },
  { "nombre": "Lola", "tipo": "Vaca", "comportamiento": { "sonido": "¡Muuu!" } }
]
```

Resumen

Clase Padre (Abstracción): Está representada por la colección `animales` en sí misma.

Define que todo lo que entre ahí debe tener una estructura base.

Herencia: Se refleja en que todos los documentos (Perro, Gato, Vaca) "heredan" la misma estructura base (tienen un nombre y un bloque de comportamiento).

Polimorfismo: Queda demostrado en el bloque `"comportamiento"`. El esquema es flexible; llamamos al mismo campo (`sonido` o `comer`), pero la base de datos nos devuelve datos completamente diferentes según el `tipo` de animal consultado.